

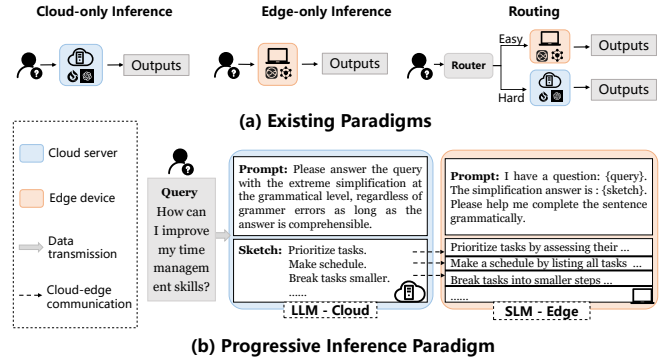
PICE: 一个语义驱动的渐进推理系统

用于云边网络中的大型语言模型服务

会友湛, 轩璋, 海生谭, *IEEE Senior Member*, 韩天, 东平永, 君阳张, 和向阳李,
IEEE/ACM Fellow

Abstract——大型语言模型（LLMs）在推动各个领域新一波交互式人工智能应用的同时，也存在高推理成本和严重依赖云端的问题。受到语言学中冗余现象的启发，我们提出了一种基于云端和边缘的渐进式推理范式，即首先在云端由LLMs生成答案草稿，然后在边缘由小模型（SLMs）进行并行扩展以填充细节。渐进式推理有望在提高吞吐量和降低推理延迟方面带来潜在优势，同时也面临关键的实现挑战，包括SLMs导致的响应质量下降、草稿简洁性与全面性之间的权衡，以及网络传输和边缘推理造成的延迟增加。在本工作中，我们提出并实现了PICE，一种具有语义级云-边协作的LLM服务系统，通过动态推理任务调度和集成学习提高推理吞吐量和质量。

会影响响应质量。为了在快速且成本效益高的推理需求之间取得平衡，提出了一种混合推理策略，利用一个路由器根据预测的查询难度，将查询动态分配到边缘的小模型或云端的大模型 [8]–[10]（图 1(a) - 右）。然而，这种粗粒度的调度方法过度依赖路由器的性能，因此需要一种更通用的方法，以有效地协调推理效率和响应质量。



一、引言

大型语言模型（LLMs），如 GPT4、LLaMA 和 Qwen [1]–[3]，在自然语言处理和聊天机器人系统中展现了前所未有的性能。然而，LLMs 的卓越能力伴随着显著的推理成本，例如 Claude [4]（通过 Slack API 访问）生成响应需要 22 秒，而 Vicuna-33B [5]（在一块 NVIDIA A100 GPU 上本地运行）生成响应需要 43 秒 [6]。如何高效地服务 LLMs 已成为基于 LLM 的 AI 应用提供良好用户体验的关键需求，因为它们的交互性质要求在推理过程中具有低响应延迟和高吞吐量。

此外，由于大型语言模型（LLM）的庞大规模，当前系统严重依赖云计算（图1(a)-左），因此面临请求负载重、运营成本高以及隐私问题[7]。为缓解这些问题，越来越多的人开始关注设计更小、成本效益更高的LLM，以便利用日益普及的众多多样化边缘计算设备。例如，Meta 提供了多种版本的 Llama [2]，仅有70亿参数，小到可以在边缘设备上运行（图1(a)-中）。尽管这种方法可以提高效率，它通常

图1: 现有推理范式与渐进式推理。

与现有在模型和系统层面优化推理的努力形成鲜明对比，我们采取了一条“语义层面”的路径，通过减少大型语言模型的输出以提高吞吐量，*i.e.*, *the LLM creates the response sketch and the SLM fills in the details*。这一观点源于人类回答问题的方式，*i.e.*, 我们只需用几个词就能传达一句话的一般意思。

受到这一想法的启发，我们提出了一种云和边缘上的渐进式推理范式（图 1(b)）。该架构利用边缘上的冗余计算能力，从云端卸载工作负载，同时确保高响应质量。具体来说，在接收到查询后，LLM 首先评估响应的潜在长度。如果响应简短，LLM 将直接输出完整答案。在预计响应较长的情况下，LLM 将生成语义完整但语法最简的草稿，然后边缘上的 SLM 对其进行扩展以提高推理效率。每个短句的语义完整性消除了其扩展的顺序依赖，使 SLM 能够并行处理每个句子，从而打破自回归模型的顺序解码。

Huiyou Zhan, Xuan Zhang, Haisheng Tan, Dongping Yong, Junyang Zhang 和 Xiang-Yang Li 隶属于中国科学技术大学（USTC）LINKE 实验室及中国科学院无线-光通信重点实验室，中国合肥 230027（电子邮件: zh anhuiyou@foxmail.com; xuanzhang@mail.ustc.edu.cn; hstan@ustc.edu.cn; yongdongping@mail.ustc.edu.cn; zhangjunyang@mail.ustc.edu.cn; xiang yangli@ustc.edu.cn）。

韩天隶属于中国合肥中国科学技术大学（USTC），邮编230027（电子邮件: henrytian@ustc.edu.cn）。

渐进推理范式虽然很有前景，但在实际实施中也带来了几个关键挑战：

1) SLM 的响应质量可能下降；2) LLM 草图的简短性与质量之间的权衡，*i.e.*，虽然更短的草图可以提高吞吐量，但所表达的语义可能不完整；以及 3) 网络传输和边缘推理造成的额外延迟。

在这项工作中，我们提出了 PICE，一种高效的 LLM 逐步推理系统，该系统协同利用云端和边缘的资源，有效减轻了云端 LLM 的工作负载，并实现了高吞吐量和优越的响应质量。PICE 的有效性来源于四个关键设计：利用逐步推理优化吞吐量的动态调度器、通过集成学习机制聚合多个小模型的输出以提升质量、加速边缘推理的 SLM 并行执行优化策略，以及能够使 LLM 生成简洁但语义完整的草图的专门微调组件。我们的主要贡献总结如下。

- 我们提出并展示了在云端和边缘进行语义层级并行数据处理时协同渐进推理的可行性。据我们所知，这是首次尝试将语义优化与云-边协作结合，以实现高效且可靠的大语言模型（LLM）服务（第 II 节）。
- 我们设计了 PICE，一种新型的用于云端和边缘大规模语言模型（LLM）服务的渐进推理引擎，通过采用动态调度策略、集成学习机制和并行边缘推理，实现了高吞吐量和低延迟（见第 III 节和第 IV 节）。
- 通过在一个由一组 Jetson AGX Orin 和云服务器组成的测试环境中实施 PICE，我们在各种大型语言模型（LLM）基准测试和流行语言模型上进行了广泛实验，结果表明，PICE 可实现 $1.5\times$ 到 $2\times$ 的吞吐量提升，并将延迟减少最多 43%，同时相比基线方法在若干问题类别上可能提高回答质量（第 V 节）。

II. 背景与动机

A. LLM Services on the Cloud and Edge

如表 I 所示，虽然大型语言模型（LLM）提供了高响应质量，但其庞大的规模需要大量计算资源，而这些资源只能由云服务器提供 [11]。随着更多基于 LLM 的服务的推出，仅依赖云计算越来越无法满足诸如吞吐量和延迟等推理效率的要求。

Model	Speed(tokens/s)	GPU Memory(GB)	MMLU
Qwen2.5-72B-Instruct	18.19	134.74	86.1
Llama3-70B-Instruct	18.82	130.64	79.5
Qwen2.5-32B-Instruct	22.13	60.11	83.3
Llama3-8B-Instruct	76.5	15.83	66.6
Qwen2.5-7B-Instruct	84.28	14.92	74.2
Qwen2.5-1.5B-Instruct	183.33	3.44	60.9

表 I：性能比较。模型的速度在使用 vLLM 框架的两块 NVIDIA A100 GPU 上进行测试。MMLU 分数是用于评估语言模型在多任务语言理解中的性能的指标 [12]。

边缘计算是一种有前景的解决方案，通过在更接近用户的边缘设备上提供服务来应对上述挑战 [13]。由于计算能力和内存的限制，部署在边缘的语言模型的规模通常远小于云端模型，这导致其响应质量下降。例如，8B 参数是 Llama 在智能手机上实现实时推理的最大规模（*e.g.*，小米 10）[14]，然而其算术推理能力显著低于 Llama3-70B 模型（表 I）。

面对推理效率和响应质量之间的权衡，最近的研究提出了一种称为路由的新推理范式，该方法在推理时使用两个不同规模的模型，而不是单一模型 [8]，[15]。较小的模型通常推理成本较低，但准确度也低于较大的模型。关键思想是识别并将简单查询路由到小模型，从而在保持响应质量的同时降低推理成本。这些方法通常需要为每种场景从头训练路由器，可能缺乏通用性。还有模型级和系统级的推理优化技术 [16]，例如模型级联 [17]、投机性解码 [18]、PagedAttention [19] 和连续批处理 [11]，[20]，这些大多适用于云服务器，并未充分利用边缘设备的计算能力。边缘智能设备的持续增长促使我们设计更通用的云端-边缘

B. Our idea: Cloud-edge Collaborative Progressive Inference

基于 Transformer 的大型语言模型（LLMs）的有效性受到计算成本相对于序列长度呈二次增长的限制，尤其是在处理长序列时。由于 LLMs 的自回归特性，生成一个 token 需要读取整个 KV 缓存。对于具有 32k 上下文长度的 Llama3-8B 模型，KV 缓存可能占用 16GB 的空间，读取至少需要 11 毫秒，这导致推理延迟超过 50%，并限制了整体吞吐量 [21]。这启发我们通过减少 LLM 的响应长度来提升推理效率。我们的想法的可行性得到以下两个观察结果的支持。

观察 1：并非所有的词元在表达语义时都起到相同的作用。正如语言学中的冗余现象，句子的语义本质通常只通过少数几个词来传达，其余的词仅用于完善语法结构。这一原则在大语言模型（LLM）的推理中也很明显。例如，已有研究指出，在情感分析任务中，表达强烈情感的词元在重要性向量中具有最高权重，而诸如“and”和“to”等介词则受到的关注最少 [17]。通常，各种模型之间的性能差异体现在这些关键词元上。如图 2 所示，Qwen2.5-72B 在“Q”、“exist”、“simultaneously”和“principle”等词元的概率分布中表现出较大的方差，而其 7B 和 1.5B 版本则相对较小。

观察 2：一旦给定关键标记，LLM 和 SLM 对其他的条件概率表现出类似的情况

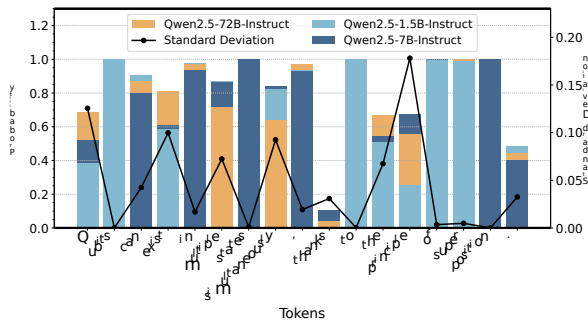


图2: Qwen2.5-72B、Qwen2.5-7B 和 Qwen2.5-1.5B 模型在不同标记上的条件概率和方差。较低的方差表示模型输出分布之间的一致性更高。

标记。我们分析了模型在生成相同答案时的条件概率分布。如图 2 所示，在提供了几个关键标记（*e.g.*、Q、exist、simultaneously）之后，模型对剩余标记的条件概率分布的方差相对较低，尤其是对于像“to”和“of”这样的介词。这表明，如果大型语言模型（LLM）可以向小型语言模型（SLM）提供关键标记，SLM 的响应质量可以与 LLM 相似。

根据上述观察，我们采用渐进推理来通过缩短大型语言模型 (LLM) 的回答来提高吞吐量。具体来说，利用对回答长度的感知 [22]，LLM 通过极端语法简化来处理长答案问题，只要答案可理解，不论语法错误与否。随后，将简化后的答案提供给边缘的 SLM 以进一步扩展为完整回答。由于每个草稿句子在语义上都是完整的，不同句子的扩展变得独立。这种独立性使得 SLM 能够并行扩展句子，从而打破自回归解码，并大大加快推理过程。根据我们的初步实验（图 3），通过使用渐进推理，LLM 的生成长度可以从平均 500 个标记减少到 200 个，从而将吞吐量提高 1.5-2 倍。

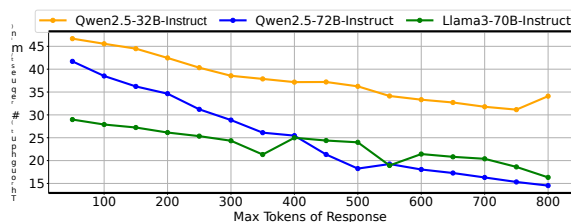


图3: 横轴表示大型语言模型 (LLM) 响应的最大令牌数, 纵轴表示服务系统的吞吐量。吞吐量以每分钟处理的查询数量 (#queries/min) 来衡量。

C. Challenges of Progressive Inference

挑战 1: 确保高响应质量。由于推理能力有限, 存在响应质量下降的风险

SLM 的能力，特别是在 LLM 提供的草图过于简略时。确保 SLM 能够在不影响准确性或连贯性的情况下，有效扩展和丰富 LLM 的框架性回应，是一个重大挑战。

挑战 2：平衡响应长度和质量。一个核心挑战是优化草图长度与质量之间的权衡。如图 3 所示，LLM 生成的较短草图会提高吞吐量，从而为 SLM 提供的参考令牌较少，这可能会降低响应质量。此问题强调了需要仔细考虑从云端传递到边缘的信息量。

挑战3: 处理由于网络传输和边缘推理引起的延迟。网络传输与边缘推理的整合增加了另一层复杂性。尽管渐进式推理只需要传输最少的文本，但网络拥堵仍可能对性能产生不利影响。此外，边缘设备有限的计算能力要求对任务进行高效的动态调度。

应对上述挑战需要努力加强模型之间的协作动态，并优化云端与边缘之间的数据流，这对应用渐进推理范式发挥其全部潜力，以实现更高效、更响应的推理系统至关重要。

III. PICE概述

受到服务大型语言模型 (LLM) 的高额开销以及渐进式推理带来的新优化机会的驱动, PICE 提出了一个新颖的云-边协作推理引擎, 以高效实现针对通用模型的细粒度吞吐量提升, 这些模型由基于实时每次查询的反馈驱动。图 4 显示了 PICE 及其工作流程的高层视图。基于云的 LLM 推理引擎评估进入用户请求的响应长度 (❶), 直接提供完整答案 (❷a), 或为复杂响应生成概要并将扩展任务排队 (❷b)。对于后者, 调度器将该任务分配给适当的边缘模型 (❸), 然后系统选择具有最高置信度的输出答案 (❹) 提供给用户 (❺)。接下来我们将详细介绍 PICE 的各个组成部分。

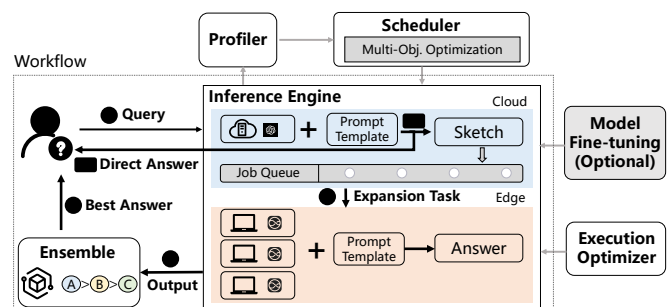


图4: PICE的概览与工作流程。

调度器。PICE 的控制部分是一个调度器，它利用渐进推理的优势来提高系统吞吐量。它会动态调整草图长度

由基于云的语言模型根据复杂的运行时条件生成，智能地在云和边缘之间分配任务以优化资源利用，并在边缘为任务执行选择最佳模型候选。

集成学习机制。为了抵消响应质量可能下降的情况，我们的系统实施了集成学习策略，其中多个较小的模型协作生成并选择最高质量的响应。这种方法利用不同模型的集体智能来提高决策准确性。

执行优化器。执行优化器旨在加速边缘设备上的推理，对于优化草图的扩展策略至关重要。它确保批次内序列长度一致，以最大化处理并行性，从而显著提高整体系统效率。

除了上述三个关键组件之外，我们的LLM服务系统中还包含其他组件。

- 分析器。它分两个阶段运行：离线阶段和运行时阶段。在离线阶段，它进行设备特定的延迟估算。在运行时阶段，它持续监控设备和服务器负载，以及网络状况，为调度器提供实时数据，以便做出明智的决策。
- 推理引擎。该引擎负责执行人工智能模型推理，分为云端和边缘部分，并结合提示模板进行渐进式推理。云端组件运行大语言模型（LLM），而边缘组件在多个设备上利用小语言模型（SLM）以实现更快速的计算和更好的响应时间。作业队列缓冲来自大语言模型的任务，然后由调度器将其优化分配给边缘模型进行处理。
- 模型微调（可选）。为了提高大语言模型生成草图的质量，我们包含了一个可选的微调组件。该组件使用来自人工智能反馈的强化学习来优化大语言模型，以生成简明的草图，能够以最少的文字捕捉完整的语义。微调是在离线环境下进行的，然后将优化后的模型部署到云端，用于云边协作的渐进式推理。

总之，我们的系统 PICE 采用了一种语义级的推理加速技术，该技术可以在不更改现有模型和硬件的情况下工作，同时也兼容诸如推测解码 [18]、PagedAttention [19] 和连续批处理 [11] 等优化技术。

四、关键设计

PICE 的核心是一种新型的渐进式推理引擎，采用高效且强大的模型来处理查询。在本节中，我们介绍了充分发挥渐进式推理潜力的关键设计。

A. Dynamic Scheduler

1) *Optimization Objective*: 我们首先提出动态调度器的优化目标，并对调度问题进行形式化。我们考虑一个由 N 对输入 x_i 及其对应输出 y_i 组成的数据集 $D = \{(x_i, y_i)\}_{i=1}^N$ 。为解决对模型的建模问题

x 到 y 的转化，我们采用了两阶段建模策略。首先，输入的 x 通过一个 LLM h 被转化为更有意义的草图 r ，表示为 $h: x \rightarrow r$ 。接下来，下一阶段是使用一个 SLM s ，它以提取的表示 r 为输入并预测输出 y ，表示为 $s: r \rightarrow y$ 。通过这种方法，我们本质上将完整的 $x \rightarrow y$ 建模操作分解为 $p(\hat{y}|x) = p_h(r|x) \cdot p_s(y|r)$ 。其中， $p(\hat{y}|x)$ 是 PICE 的概率模型，作为 $p(y|x)$ 的近似，误差 $\delta(\hat{y}, y)$ (δ 是误差函数)。

因此，我们的目标之一是通过 SLM s 和草图 r 的双重优化来减少由 $E[\delta(\hat{y}, y)]$ 表示的误差：

$$\sigma^* = \arg \min_{s \in S, r \in R} E[\delta(\hat{y}, y)] = \arg \min_{s \in S, r \in R} E[\delta(s(h(x)), y)] \quad (1)$$

其中 σ^* 表示在集合 S 中的所有模型和所有可能的草图 R 上的最优 SLM 和草图。

系统的另一个目标是最大化吞吐量。我们定义云端大型语言模型（LLM）处理长度为 l 的响应的平均时间为 $f(l)$ 。设 c 为成本系数，其定义为在云端执行一次 LLM 与在边缘执行 SLM 所需时间的比值。 c 的值不仅依赖于模型，还取决于硬件和软件的具体情况。因此，边缘 SLM 生成长度为 l 的响应的平均处理时间为 $c \cdot f(l)$ 。吞吐量可表示为 $\max(f(|r|), \Delta(r), \frac{c}{p} \cdot f(l))$ ，其中 $\Delta(r)$ 是边缘与云之间的网络延迟， p 表示边缘推理的并行度。其他约束条件包括保持每个请求的处理延迟低于 $f(l)$ ，即云端推理的延迟。

为了支持现实的多标准服务水平协议（SLA），我们将调度问题建模为多目标优化问题。当前的指标集定义为

$$M = \{\text{错误, 吞吐量, 延迟, 服务器成本, 边缘成本}\}$$

我们分别根据生成的令牌数量定义服务器成本和边缘成本。类似于 SPINN [23]，我们将上述目标分类为硬约束，例如推理延迟，以及软约束，例如最大化吞吐量和最小化错误。为了让用户能够指定每个指标的重要性，我们采用了多目标字典序公式 [24]：

$$\min_{\sigma} M_i(\sigma), \quad s.t. \quad M_j(\sigma) \leq M_j(\sigma_j^*) \\ j = 1, 2, \dots, i-1, \quad i > 1, \quad i = 1, 2, \dots, |M|$$

其中 $M_i \in \{M_1, M_2, \dots, M_{|M|}\}$ 是软目标有序元组中的第 i 个指标， i 是该指标在重要性序列中的位置， $M_j(\sigma_j^*)$ 表示在第 j 次迭代中找到的第 j 个指标的最优值。在此公式下，我们根据目标使用案例的要求对指标进行重要性排序。

2) *Cloud-side scheduling*: 我们通过多个步骤来解决这个优化问题。在云端进行大型语言模型推理时，我们首先确保满足推理延迟的硬性约束，同时提高吞吐量和

响应质量。云端调度主要通过动态调整草图的长度来实现这一点。

具体而言，一个查询的端到端总延迟包括来自大语言模型（LLM）推理、等待时间、网络传输和小语言模型（SLM）推理的延迟组成部分。我们将 LLM 推理延迟定义为输出长度 f 的函数，使得任务 r_i 在 SLM 上的推理延迟由 $\frac{c \cdot f(l_i)}{p}$ 给出，其中 l_i 表示该查询的预期响应长度。等待时间表示边缘上作业队列 Q 中所有任务的总完成时间，计算公式为 $\frac{\sum_{r_j \in Q} c \cdot f(l_j)}{Np}$ ，其中 N 表示边缘设备的数量。因此，端到端延迟的约束可以表示为如下公式：

$$f(|r_i|) + \Delta(r_i) + c \cdot f(l_i) + \frac{\sum_{r_j \in Q} c \cdot f(l_j)}{pN} \leq f(l_i) \quad (2)$$

我们的调度算法的灵感来自于这样的观察：类似于人类能够根据对查询的理解来估计答案的长度，大型语言模型（LLM）也可以提前感知回答的长度，这一点已被以前的工作[22]所证明。在我们的工作流程中，LLM 会同时输出一个草稿和预期的答案长度 l_i 。在我们的工作流程中，LLM 会输出一个草稿和预期的答案长度 l_i 。利用我们在离线阶段通过分析工具建立的 f 和 c 的测量值，我们可以估计边缘推理延迟和等待时间。

我们定义了多个草图长度等级，从 0 到 l_i 。在收到查询时，我们首先评估网络的条件，并保守地估计边缘推理延迟，默认将 $p =$ 设置为 1。如果没有高于 0 的等级满足不等式 (2) 设定的条件，我们将放弃渐进式推理，直接向 LLM 请求完整响应。如果条件允许，我们会评估 SLM 选择合适草图长度的能力，更有能力的 SLM 可能会选择较短的长度以提高吞吐量。请注意，由于草图长度是通过 LLM 提示指定的， r_i 可能与预期值相差多达 10 个令牌。

3) *Job dispatching*: 在处理需要渐进推理的查询的过程中，至关重要的是以确保负载均衡的方式在边缘设备之间分配扩展任务。为了管理这些工作负载的动态性质，我们实现了一种多列表调度算法，根据不断变化的需求自适应地分配推理任务。

在实际场景中，对不同查询的响应长度可能存在显著差异。这种在批量 SLM 推理时的可变性会引入低效，因为包含了长度不同的序列。具体来说，较短的序列必须等待较长序列完成，从而导致计算延迟。推理的二次时间复杂度进一步加剧了这些低效，使该过程负担沉重。

我们的作业调度算法，如算法1所述，主要是一种多列表调度算法，根据预期答案长度对任务进行分类 l_i 。当查询进入作业队列时 Q ，它们会根据长度分配到特定列表（第2-6行）。一旦边缘设备被

空闲时，它们会主动从最长的列表中取出一批任务来执行推理（第7-11行）。

Algorithm 1 Job Dispatching

```

1: Initialize lists  $q_1, q_2, \dots, q_n$  for different lengths.
2:  $Q \leftarrow$  List of jobs  $(r_i, l_i)$  waiting for processing
3: while  $Q$  is not empty do
4:   Remove task  $r_i, l_i$  from  $Q$ 
5:   Determine list index  $j$  based on  $l_i$ 
6:   Add  $r_i$  to  $q_j$ 
7: for each edge device  $D$  do
8:   while  $D$  is idle do
9:      $max \leftarrow$  List with the most jobs
10:     $batch \leftarrow$  SelectBatchFrom( $q_{max}$ )
11:     $D.ProcessBatch(batch)$ 
```

4) *Edge-side scheduling*: 为了应对效率与响应质量之间的权衡，调度器必须在边缘设备上为推理选择合适的 SLM。模型选择过程包括离线分析和在线选择。分析会生成一系列不同优化的候选模型。当任务到达时，PICE 会选择一個能够以最低延迟满足目标的候选模型。

离线分析。候选模型的性能由基础模型和硬件共同决定。我们最初使用分析器评估候选 SLM 和 LLM 在边缘设备和云服务器上生成指定长度序列时的推理延迟，并计算其成本系数 c 。随后，我们测试 LLM 在云服务器上生成不同长度序列时的推理延迟。两者结果都用于估算作业队列中任务的执行时间。

在线候选选择。如算法2所示，当边缘设备从作业队列中获取任务 r_i 时，它首先根据大模型（LLM）预期的响应长度 l_i 和离线分析数据估算剩余处理时间（第2行）。如果任务的总推理时间超过阈值 $f(l_i)$ ，从而违反优化器的硬约束，则必须选择较小的中型模型（SLM）（第3~4行）。相反，如果总推理时间低于阈值，则可以选择更高质量的 SLM 进行推理，或者保留当前的 SLM。为了避免频繁切换模型带来的开销，后者过程仅在作业队列中待处理任务数量少于其最大值时进行（第6~12行）。

B. Execution Optimizer

尽管边缘设备上的模型尺寸减小，但受限于其计算和内存能力的推理延迟，仍可能对整体系统构成显著瓶颈。因此，加快边缘设备上的推理过程至关重要。

我们通过语义层面的并行策略优化边缘推理。具体来说，边缘推理引擎接收到的草稿由多个简短的句子组成。由于每个简短的句子在语义上是完整的，它们的扩展是

Algorithm 2 Model Selection

```

1: Input:  $r_i, l_i, f(l_i), f(|r_i|)$ 
2: Estimate remaining processing time  $\tau_i$  for current SLM.
3: if  $\tau_i > f(l_i) - f(|r_i|)$  then
4:   Switch to a smaller SLM for  $r_i$ 
5: else
6:   if  $|JobQueue| < \text{maximum}$  then
7:     for each SLM  $m_j$  larger than the current SLM do
8:       Calculate  $\tau'_i$  for  $r_i$ 
9:       if  $\tau'_i < f(l_i) - f(|r_i|)$  then
10:        Upgrade to  $m_j$  for  $r_i$  and break the loop
11:   else
12:     Continue with current SLM for  $r_i$ 

```

彼此独立，因此可以并行执行。提示模板如下。

[系统]: 我有一个关于 {查询} 的问题。简化的答案如下: {草图}。现在, 请帮我完成并且只完成一个简短句子的书写 {句子}。不要继续写其他句子! [助手]:

然而，更高的并行性并不总是更可取。这是由于两个因素。(1) 句子长度的可变性。句子长度的差异可能导致在简单批处理时出现不必要的等待。(2) 提示开销。需要额外的草图信息可能导致SLM需要处理更长的提示。过度的并行性可能导致KV缓存中出现冗余的草图信息，从而降低系统效率。

为应对这些挑战，我们采用基于二叉树的短句合并策略。考虑一个包含 k 个短句的草图 r_i ($r_{i,1}, r_{i,2}, \dots, r_{i,k}$)，我们假设短句中的词数与其扩展后的词数呈正相关。因此，我们按词数对 k 个短句进行排序，并将它们合并为 $\lceil \frac{k}{2} \rceil$ 组，如下所示: ($r_{i,1}, r_{i,k}$), ($r_{i,2}, r_{i,(k-1)}$), $\dots$, ($r_{i, \lceil \frac{k+1}{2} \rceil}, r_{i, \lceil \frac{k+1}{2} \rceil}$)。如果当前的并行度仍然可以满足推理延迟的硬性约束，我们会递归地应用上述合并过程，以提高边缘设备的吞吐量。

C. Ensemble Learning

开源的SLM由于数据、架构和超参数的差异，表现出多样的优势和劣势，使它们彼此互补。这种多样性促使我们动态集成这些SLM，以便为每个输入生成始终更好的响应。典型的集成方法包括从不同模型生成的输出中选择最优候选，利用如MLM-Scoring [25] 和 BERTScore [26] 的评估指标来训练奖励模型 [27]。然而，由于训练和部署奖励模型带来的额外开销，这些方法不适用于我们的系统。为了避免

由集成造成的开销，一个直接获得置信度的方法是使用困惑度 [28]，它是一种评估模型理解和生成类人语言能力的指标。困惑度的计算方法是对一个词序列的平均负对数似然取指数。

然而，我们在实践中观察到，困惑度过度依赖于模型本身。例如，Llama3-8B 模型在所有查询中始终表现出比 Qwen2.5-7B 更高的困惑度，而不管回答的实际质量如何。为了全面评估模型及回答的质量，我们使用文本评分与困惑度的加权平均作为置信度。文本评分由两个部分组成：其一是答案的长度，SLM 对答案的更详细扩展会获得更高的评分；其二是答案与草稿的相似度，相似度越高得分越高，用 rouge-l 分数[29]表示。因此，序列的置信度可以按如下方式计算。

$$\text{con}(\hat{y}) = \alpha_1 \cdot 2^{\frac{1}{N} \sum_{i=1}^N \log_2 p(w_i)} + \alpha_2 \cdot \text{Norm}(|\hat{y}|) + (1 - \alpha_1 - \alpha_2) \cdot \text{Rouge-l}(r, \hat{y}) \quad (3)$$

其中 N 是令牌的总数， w_i 是答案中的第 i 个令牌， $p(w_i)$ 是模型分配给第 i 个令牌的概率。参数 α_1 和 α_2 是权重，而 $\text{Norm}(|\hat{y}|)$ 表示响应长度的归一化。为了减轻采样策略的影响，单个 SLM 可能生成多个序列。

D. Model Fine-tuning

PICE中的模型微调组件是一个可选模块，旨在增强大语言模型（LLM）生成简明草图的能力，使其以最少的文字传达完整的语义。与典型的基于人类反馈的强化学习（RLHF）不同，该组件引入了一种专门的偏好标注算法，专门用于优化模型生成高质量草图的能力。如图5所示，微调过程包括三个步骤：监督微调、奖励模型训练和基于强化学习的微调。

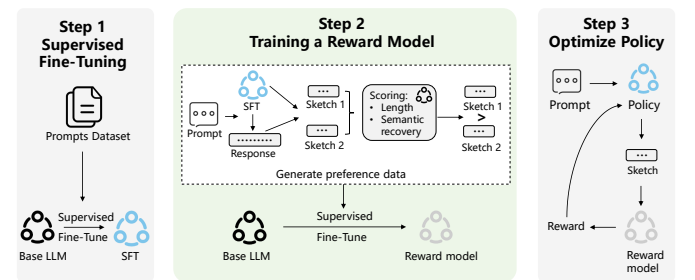


图5: PICE微调组件的设计。

在第一步中，预训练的大型语言模型（LLM）在标注数据集上进行微调，以生成简明的草图（例如，给定一个输入文档，生成一个抓住文档主要思想的摘要），使用逐令牌监督来生成一个监督微调（SFT）模型 π_0 。

在第二步中，我们的目标是生成一个奖励模型，该模型接收一系列草图，并返回一个标量奖励，该奖励应数值上表示我们对优秀草图的偏好。奖励模型的提示-生成对训练数据集由SFT模型和偏好标注算法生成。给定输入 x ，PICE首先从SFT模型 π 获取完整答案 y 。然后，将查询 x 和答案 y 提供给模型 π ，该模型生成一对草图 $(r1, r2)$ 用于 y 。输入和草图会被发送到偏好标注算法，以根据以下标准评定哪幅草图更好。

- 草图 (l_r) 的长度越短，得分就越高。
- 基础大语言模型将草图 r 扩展为完整答案 $\hat{y}$ ，然后与 SFT 模型的答案 y 进行比较。相似度越高，草图的得分越高。

草图的得分可以按如下方式计算： $\beta_1 \cdot \frac{1}{l_r} + \beta_2 \cdot \text{Rouge-l}(\hat{y}, y)$ 。参数 β_1 和 β_2 是权重。这些元素形成一个三元组数据集 $D = \{(x, r_w, r_l)\}$ ，其中 r_w 和 r_l 分别是优选（即得分较高）和非优选的草图。奖励模型（RM） R_ϕ 通过最小化以下损失进行训练：

$$\mathcal{L}_R(\phi) = -\mathbb{E}_{(x, r_w, r_l) \sim D} [\log \sigma(R_\phi(x, r_w) - R_\phi(x, r_l))],$$

其中 σ 是 sigmoid 函数。

在第三步中，策略 π_θ^{RL} 使用 SFT 模型的权重进行初始化，然后通过强化学习进行优化，以最大化来自 RM 的奖励，RM 充当草图偏好的代理。为了防止与原始 SFT 策略 π 产生过度偏离，目标函数中引入了 Kullback-Leibler (KL) 散度 [30] 项 D_{KL} 。该项由超参数 γ [31] 控制，以确保 π_θ^{RL} 不会产生尽管 RM 给了高奖励但质量低或语言不自然的输出 [32]。优化目标定义如下：

$$J(\theta) = \mathbb{E}_{r \sim \pi_\theta(\cdot|x)} [(1 - \gamma)R_\phi(r|x) - \gamma D_{KL}(\pi_\theta^{RL}(r|x) \parallel \pi(r|x))].$$

五、评估

我们通过基于测试平台的大量实验，使用各种大型语言模型基准和流行语言模型来评估 PICE。我们首先评估其在提升推理效率和保证响应质量方面的整体表现（第 V-A 节）。接着，我们在云-边缘渐进推理框架中检查其三个核心组件（第 V-C 节）。此外，我们分析了超参数对系统性能的影响（第 V-D 节）。主要结论如下：

- PICE 相比基线方法实现了 $1.5 \times \sim 2 \times$ 的吞吐量提升，并将延迟最多降低 43%。
- PICE 有效地保持了响应质量，甚至在若干问题类别上提升了响应质量。
- LLM 服务系统内的战略优化使 PICE 能够在各种场景下有效平衡推理效率和质量。

A. Methodology

测试平台设置。我们的实验基础设施包括八台设备，其中包含四台 Jetson AGX Orin 单元和一台配备四块 NVIDIA A100 Tensor Core GPU 及 112 个 CPU 核心的云服务器，这些设备作为主要的推理设备。表 II 提供了这些设备的详细规格。在软件环境方面，云端推理引擎运行在 vLLM 框架 [19] 上，而边缘推理引擎由 PyTorch [33] 和 Transformers [34] 实现。

	Device	Memory	Memory bandwidth	AI performance
Edge	Jetson AGX Orin	64GB	204.8GB/s	137.5 TFLOPS
Cloud	NVIDIA A100	80GB	1935GB/s	624 TFLOPS

表 II：物理设备。

指标与基准。通过一系列 Llama 和 Qwen 模型评估 PICE 的性能，如表 I 所示。推理效率的关键指标是吞吐量，定义为系统每分钟处理的请求数量。此外，我们还测量平均端到端延迟，包括云端和边缘的推理延迟、等待时间以及数据传输延迟。为了评估响应质量，我们采用两个基于 LLM 的评估框架：FastChat [35] 和 LLMZoo [36]。这些框架涉及向 LLM 评审（例如 GPT-3.5 turbo）提供一系列开放式问题，并根据质量对响应进行评分和排名。我们特别使用两个数据集，MT-bench 和 Vicuna-bench [35]，用于评估聊天机器人的指令执行能力，并根据其核心能力（e.g.、推理和数学）进行区分。

基线。我们将 PICE 的推理效率和答案质量与基线进行比较。

- 仅限云端。所有查询均由部署在云端的基于 vLLM 框架 [19] 的语言模型处理。
- 仅限边缘。在这种情况下，所有查询都由部署在边缘设备上的 SLM ($\leq 8B$) 本地处理。每个边缘设备都承载一个模型，并且查询以负载均衡的方式分发。
- 路由 [8]。它使用路由器根据预测的查询难度，将查询动态分配到边缘的小模型或云端的大模型。

B. Overall Performance

吞吐量和延迟。为了评估吞吐量，我们将云服务器上 Qwen2.5-72B-Instruct 模型的最大批量大小配置为 20，其他设备和模型按比例进行调整。每分钟请求数 (RPM) 为最大批量大小的 1.5 倍。LLM 推理的吞吐量和端到端延迟如表 II I 所示。我们有以下观察结果。首先，PICE 对大型语言模型部署具有潜力和益处。PICE 利用边缘计算资源实现了显著更高的推理吞吐量和更低的推理延迟，相较于基线方法。对于 Llama3-70B 模型，PICE 提供的平均吞吐量为每

Methods	Qwen2.5-72B-Instruct		Llama3-70B-Instruct		Qwen2.5-32B-Instruct		Llama3-8B-Instruct		Qwen2.5-7B-Instruct		Qwen2.5-1.5B-Instruct	
	Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency	
Cloud-only	14.891	138.62	16.33	121.54	32.13	72.32	75.51	28.57	88.33	30.88	148.12	23.71
Edge-only	OOM	OOM	OOM	OOM	OOM	OOM	6.03	804.21	6.68	801.23	21.20	210.38
Routing	14.86	145.04	13.79	143.94	30.04	88.57	69.55	74.75	69.55	68.66	133.31	41.28
PICE	21.24	97.34	25.98	75.15	34.81	61.22	70.48	30.21	84.98	31.78	140.86	26.19

表 III: 推理效率比较。表中所指的模型为在云端运行的模型。对于路由和 PICE, 边缘的 SLM 是指参数少于云端模型的任意模型。吞吐量以每分钟处理的查询数量 (#queries/min) 来衡量, 延迟以秒 (s) 为单位。“OOM”表示“内存不足”。

Methods	Metrics	Overall	Question categories									
			generic	knowledge	roleplay	fermi	coding	math	writing	reasoning	stem	humanities
Cloud-only	Overall score	8.03	7.90	8.20	8.12	8.20	8.00	8.50	7.81	8.33	8.20	7.83
	Diversity rank	1.82	1.90	1.80	1.59	1.50	1.50	1.36	1.56	1.67	2.50	2.78
	Relevance rank	1.67	1.40	1.00	1.06	1.10	1.00	1.00	1.17	1.00	4.00	4.00
	Immersion rank	1.60	1.60	1.00	1.35	1.10	1.25	1.09	1.17	1.00	2.90	3.56
	Coherence rank	1.55	1.30	1.00	1.18	1.00	1.25	1.00	1.17	1.17	3.60	2.89
	Integrity rank	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Edge-only	Overall score	7.38	8.00	7.60	7.00	7.60	7.27	7.70	7.75	6.83	7.60	8.00
	Diversity rank	1.81	1.70	1.60	1.88	1.40	1.81	2.18	2.28	2.67	1.40	1.22
	Relevance rank	2.01	1.20	1.00	2.24	1.60	2.63	3.36	2.50	3.50	1.10	1.00
	Immersion rank	1.99	1.20	1.20	2.41	1.40	2.44	2.82	2.33	3.83	1.20	1.11
	Coherence rank	1.90	1.60	1.00	1.88	1.40	2.25	2.64	2.22	3.83	1.00	1.22
	Integrity rank	1.44	1.00	1.00	1.38	1.00	1.56	2.36	1.83	2.33	1.00	1.00
Routing	Overall score	7.52	7.70	7.70	7.82	7.90	7.67	8.30	8.00	6.67	7.80	7.33
	Diversity rank	1.81	2.00	2.10	1.76	1.50	1.94	2.09	1.67	2.33	1.40	1.56
	Relevance rank	1.71	1.20	1.30	1.47	1.70	2.00	2.36	1.67	2.33	1.20	1.44
	Immersion rank	1.90	1.60	1.90	1.47	2.20	1.81	3.00	1.56	2.83	1.50	1.22
	Coherence rank	1.75	2.40	1.00	1.24	1.90	2.06	2.36	1.28	2.83	1.20	1.44
	Integrity rank	1.32	1.00	1.00	1.00	1.30	1.31	1.27	1.00	2.00	1.60	1.33
PICE	Overall score	8.37	8.30	8.60	8.65	8.60	7.00	8.10	8.12	9.00	8.80	8.50
	Diversity rank	1.70	1.40	1.70	2.00	2.10	1.44	1.18	1.39	2.00	2.20	2.44
	Relevance rank	1.10	1.00	1.00	1.12	1.10	1.00	1.00	1.11	1.67	1.50	1.56
	Immersion rank	1.21	1.20	1.00	1.59	1.10	1.13	1.09	1.11	1.67	1.20	1.00
	Coherence rank	1.13	1.20	1.00	1.18	1.30	1.13	1.00	1.11	1.67	1.40	1.00
	Integrity rank	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

表 IV: 响应质量比较。总评分由 FastChat [35] 提供, 评分范围为 (1-10), 分值越高表示质量越高。除了整体指标外, L LMZoo 提供了五项详细指标 [36]。对于相同的问题, LLMZoo 对四种方法提供的回答在这些指标上进行排名, 排名范围为 (1-4), 其中排名 1 表示表现最佳。两个评估框架均使用 GPT-3.5 Turbo 作为 LLM 评判。

每分钟, 大约是仅云方案的1.59倍, 是路由方法的2.32倍, 同时延迟也降低了约38%到58%。对于Qwen2.5-72B模型也有类似的改进。其次, 对于Qwen2.5-32B模型, PICE的性能与仅云方案相当。这归因于该模型预测响应长度的能力较差, 经常低估响应长度, 导致PICE倾向于不启用渐进推理模式。第三, 对于像Llama3-8B这样的小模型, PICE的效率略低于仅云方案, 但优于路由和仅边缘方案。这是因为小型边缘模型与云模型之间的规模差异不足, 使边缘推理成为系统的瓶颈。因此, 我们建议PICE中LLM的规模应超过SLM的十倍以上。

响应质量。如表 IV 所示, PICE 的整体得分高于基线方法, 这表明由于整合了多个模型的输出, PICE 提高了响应质量, 相比单独使用大型语言模型尤其如此

对于知识、角色扮演和推理问题, PICE表现不错。然而, 由于难以在LLM生成的初步草图中完全捕捉核心含义, PICE在数学和编码任务上的表现不如仅云方法。此外, PICE在连贯性、多样性、沉浸感、完整性和相关性指标上均优于基线方法, 取得了最佳平均排名, 尤其是在所有任务中的完整性方面。这一表现表明, 逐步推理的两阶段方法虽然依赖于LLM生成草图的质量, 但能够有效保持完整性并促进输出的多样性。

C. Performance Breakdown

我们分别评估了 PICE 关键设计所带来的收益。实验在云端使用 LLama3-70B-Instruct 进行, 在边缘使用三个 SLM 进行。通过动态调度器提高效率。PICE 中的动态调度器通过适应运行时条件, 在推理效率和响应质量之间实现平衡。相比之下,

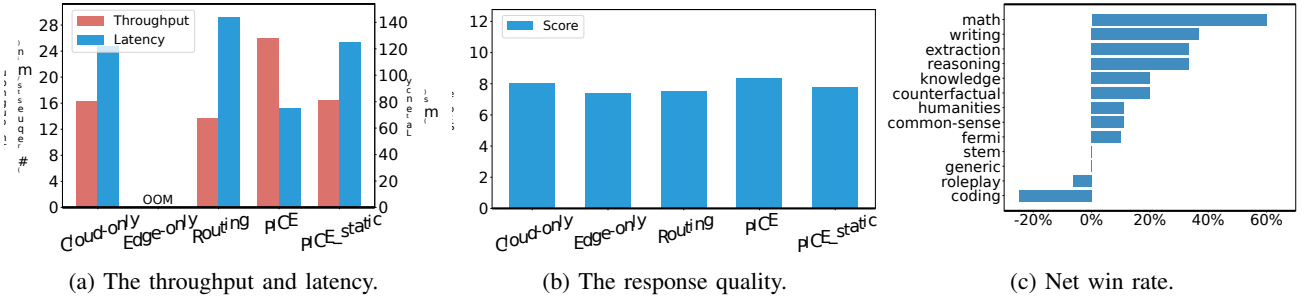


图6: 使用动态调度器时效率和响应质量的提升。(c) 中的净胜率是PICE相对于使用静态调度器时答案更好和更差的问题比例的差值。

静态调度遵循预定义的规则和参数, 仅根据预测的答案长度在云端和边缘设备之间分配请求。如图6(a)所示, 与仅云端方法相比, 采用静态调度的PICE稍微提升了吞吐量, 而动态调度则显著将吞吐量提高了额外50%。此外, 与静态调度及其他方法(如仅云端或路由)相比, 动态调度显著降低了延迟, 证明其在平衡吞吐量和响应时间方面的有效性。

如图6(b)所示, 响应质量提高了6.0%, 甚至超过了独立大语言模型(LLM)的表现, 因为PICE能够生成更详细和更全面的答案。如图6(c)所示, 动态调度器显著提高了69%问题类别的答案质量, 在数学和写作任务中表现尤为突出, 净胜率分别达到60%和36.8%。

使用执行优化器降低延迟。执行优化器利用语义级并行性来加速边缘推理。在图7(a)中, 我们根据草图长度分析了不同任务的最佳并行度。对于通用和角色扮演等任务, 并行度随草图长度增加, 在大约500个标记时达到峰值, 然后下降, 这表明随着草图变长可以获得好处, 直到受到边缘设备内存的限制。相反, 常识和数学等任务保持始终较低的并行度, 因为它们本质上较短的响应不需要高并行度。

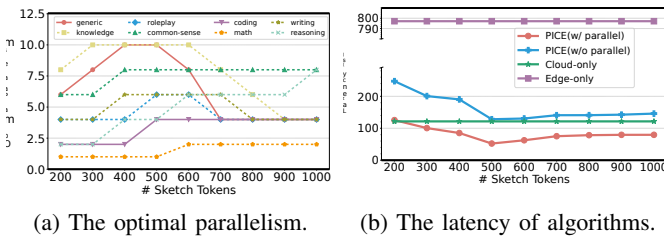


图 7: 并行生成的影响。

如图7(b)所示, 并行机制至少将PICE系统的推理延迟降低了62秒。当草图令牌数少于500时, 增加草图长度可以增强边缘推理的并行性, 从而显著降低延迟。然而, 一旦令牌数超过500个, PICE的延迟降低效果就不那么明显了。这是因为所需的KV缓存对于

提示增大, 而边缘设备的有限内存无法支持进一步增加并行度。这些结果强调了根据任务类型和草图调整并行度以优化推理效率的必要性。

通过集成学习提升质量。如图8所示, 不同问题类别下SLM的置信度排名差异显著, 凸显了每个模型的专业能力。利用这种多样性, PICE采用集成学习方法来选择最佳答案。如图9所示, 该方法在几乎所有类别中提高了回答质量, 除了编码和反事实情境, 其中角色扮演(13.95%)和知识(11.69%)类别的提升最为显著。

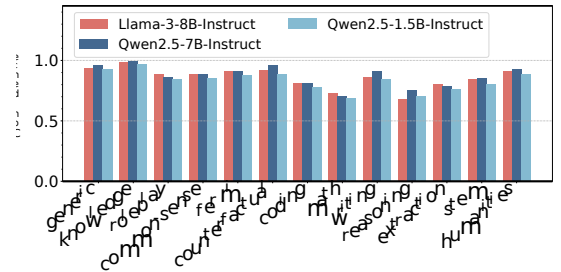


图 8: SLM 的置信度。

此外, 整体响应质量提高了2.8%, 这表明集成学习在结合各个模型的优势方面是有效的。在角色扮演和知识等类别中的显著提升凸显了集成模型在更好地捕捉细腻和详细信息方面的能力, 从而确保在需要多样化模型能力的任务中输出更高质量的结果。

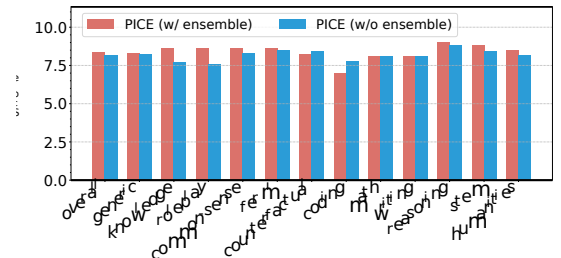


图9: 集成学习对响应质量的影响。

通过模型微调提高性能。Th

实验表明，微调组件对模型生成简洁且语义完整的草图的能力具有影响。如图10所示，与基础模型相比，微调模型在多个类别中实现了草图长度的缩短。例如，写作类别显著从52.3降至42.6，知识类别从36.9降至27.7。相反，诸如反事实（从25增加到26.4）和通用类别（从18.7增加到20.7）的轻微增长表明，微调过程在这些特定领域保持甚至提升了性能，这可能是通过生成更长的草图来表达语义实现的。

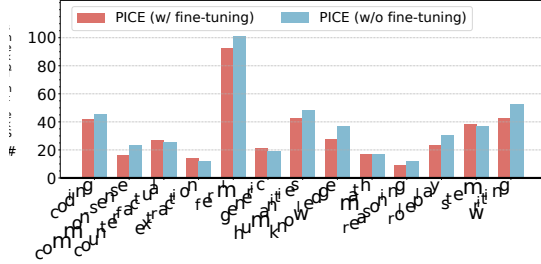


图10：各类别草图的长度。

图11所示的结果展示了在应用基于AI反馈的强化学习前后，模型的响应质量。结果表明该算法的有效性，经过微调的模型在通用、数学和推理等类别中获得了更高的评分，表明响应质量有所提高。然而，在知识和写作方面，微调后的模型表现略差，这很可能是由于其专注于生成简洁概述，偶尔会牺牲关键的语义细节。这突出了简洁性与完整性之间的权衡，尤其对于需要详细解释的任务。

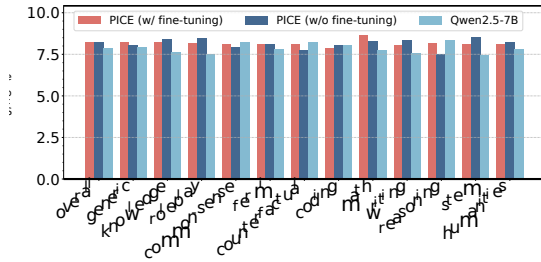


图 11：微调组件对各类别响应质量的影响。

D. Sensitivity Analysis

我们在不同条件下进行实验，包括不同的每分钟请求数（RPM）、作业队列长度和网络环境，以全面评估 PICE 的鲁棒性和适应性。

RPM的影响。图12说明了不同RPM对系统性能的影响。当RPM低于20时，这对应于云中LLM支持的最大批量大小，PICE的性能与仅云方法相当。然而，当RPM超过20时，

PICE 通过将任务卸载到边缘设备来持续提高吞吐量，而仅云端的吞吐量保持不变，且由于等待时间更长，其端到端延迟急剧增加。虽然路由方法试图将请求动态分配到边缘设备，但其效率受限于边缘可用资源的限制。

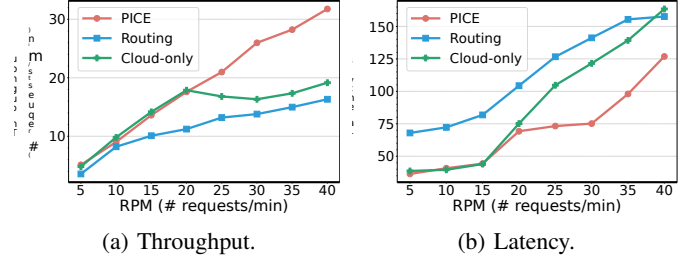


图12：RPM（每分钟请求数）的影响。

作业队列的影响。作业队列的配置显著影响 PICE 的调度策略。如图 13 所示，当队列长度设置为四时，可以实现最佳吞吐量，这允许每个边缘设备处理一个待处理请求，而基于云的 LLM 处理剩余工作负载。当队列长度超过八时，等待时间显著增加，从而导致整体延迟大幅上升。这凸显了将作业队列参数与边缘设备数量及边缘推理延迟相匹配的重要性。对这些参数进行适当调整，可以确保资源的高效利用并最小化延迟，从而提升整体系统性能。

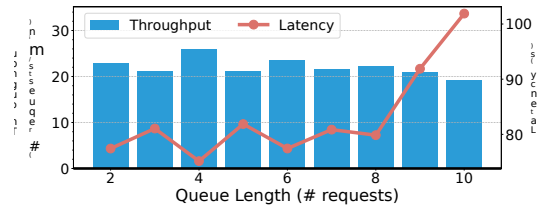


图13：作业队列长度的影响。

带宽的影响。图14展示了云端与边缘设备之间不同带宽对 LLM推理系统的延迟和吞吐量的影响。在图14中，在不同带宽条件下分析了吞吐量。PICE方法在所有带宽水平下的吞吐量始终高于仅云端和路由方法。这表明PICE在利用边缘资源卸载任务并维持性能方面的高效性。仅云端方法随着带宽增加改进有限，凸显了其对集中资源的依赖。与此同时，路由方法由于任务分配不优化而表现不佳。

图14显示，由于动态调度和任务卸载，PICE在所有带宽下都实现了最低的延迟。在所有方法中，包括PICE、仅云以及路由，延迟随着带宽增加的变化都很小。这是因为主要的计算开销在于推理，而云与边缘设备之间的数据传输——仅限于查询或草图——仅需很少的时间。

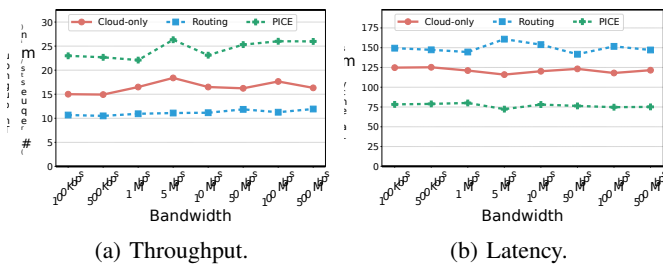


图14: 带宽对推理效率的影响。

即使在较低带宽下,也只有几十毫秒。这些结果表明,网络带宽对整体系统性能的影响很小,推理时间是主要因素。

六、相关工作

我们回顾了关于大语言模型 (LLMs) 的研究,涉及响应质量、推理效率以及两者之间的平衡。响应质量。Ning *et al.* 提出了SoT [6], 指导LLMs先生成答案的骨架,然后通过并行API调用填充内容。然而,SoT的适用性受其并行机制的限制。当查询涉及点与点之间的渐进关系时,例如创建详细描述巴黎景点和沿途美食的旅行指南,SoT在提供有效答案方面可能会遇到局限性。此外,研究表明,在多任务学习和复杂数据处理过程中,集成模型显著提升了响应质量。例如,[37]提出了一种将指令调优与稀疏MoE技术相结合的方法,有效增强了语言模型的表达能力,同时提升了准确性和计算效率。受这些工作的启发,我们使用LLM生成

推理效率。在系统层面,推测解码方法 [14]、[18]、[38] 采用 SLM 来生成一些连续的序列 token, 并并行应用目标 LLM 来验证它们, 以打破 LLM 的顺序解码。Kwon *et al.* 提出了一种称为 PagedAttention 的新型内存管理机制, 并建立了高吞吐量系统 vLLM [19]。还有一些其他工作探索了 LLM 的分布式部署以提高效率, 例如 EdgeShard [7] 和 PrivateLoRA [39]。具体来说, [7] 提出了 EdgeShard 框架, 将大型语言模型 (LLM) 划分为多个段, 并利用动态规划算法优化它们在分布式设备上的部署, 从而提升 LLM 推理的效率。在模型层面, 采用了模型量化 [40]-[43]、剪枝 [44] 和蒸馏 [45] 等方法来实现模型轻量化。这些方法旨在减少模型的参数

通过设计和使用模型压缩技术, 例如知识蒸馏和切片, 该系统将 MoE 模型的大小缩小了 3.7 倍。

效率与质量之间的平衡。Tabi 将 SLM 与 LLM 集成, 以在保持分类任务高准确率的同时减少推理延迟 [17]。对于生成任务, 提出了一种混合推理策略, 该策略使用路由器根据预测的查询难度动态地将查询分配到边缘的小模型或云中的大模型 [8], [9], [47]。DejaVu 使用每一层的输入来预测上下文稀疏性, 并采用异步和硬件感知的方法来加速推理, 同时确保模型质量 [48]。

七、结论

我们提出了 PICE, 这是一种用于云端和边缘的大型语言模型 (LLM) 服务的渐进式推理系统, 旨在有效平衡效率和响应质量。我们通过动态调度器确保 PICE 的稳健性能, 该调度器利用边缘计算优化吞吐量; 通过集成学习机制聚合多个小模型的输出以提升质量; 通过执行优化器利用语义层面的并行加速边缘推理; 以及通过模型微调策略优化 LLM, 使其生成简明且语义完整的草图。实验证明, PICE 在不影响响应质量的情况下, 实现了吞吐量提高 1.5 – 2× 和延迟降低 43%, 为 LLM 应用的广泛部署提供了可行的解决方案。PICE 是面向语义的推理效率优化的初步尝试, 并展示了通过显式规划答案结构来引出高质量答案的潜力。

参考文献

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 技术报告”, *arXiv preprint arXiv:2303.08774*, 2023.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: 开源基础模型及微调聊天模型”, *arXiv preprint arXiv:2307.09288*, 2023.
- [3] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, “Qwen 技术报告”, *arXiv preprint arXiv:2309.16609*, 2023.
- [4] Anthropic, “介绍 Claude”, <https://www.anthropic.com/index/introducing-claude>, 2023年5月.
- [5] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, “Vicuna: 一款以 90%* ChatGPT 质量令人印象深刻的开源聊天机器人”, *See https://vicuna.lmsys.org (accessed 14 April 2023)*, 第 2 卷, 第 3 期, 第 6 页, 2023.
- [6] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, 和 Y. Wang, “思维骨架: 提示大型语言模型进行高效并行生成”, 发表于 *The Twelfth International Conference on Learning Representations*, 2024.
- [7] M. Z

- [10] X. Zhang, Z. Huang, E. O. Taga, C. Joe-Wong, S. Oymak, 和 J. Chen, “Treacle: 通过上下文感知的大语言模型和提示选择进行节俭推理”, *arXiv preprint arXiv:2404.13082*, 2024.

- [11] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim 和 B.-G. Chun, “Orca: 用于基于Transformer生成模型的分布式服务系统”, 发表于 *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*, 2022, 第 521–538 页。[12] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song 和 J. Steinhardt, “测量大规模多任务语言理解”, *arXiv preprint arXiv:2009.03300*, 2020。[13] H. Hua, Y. Li, T. Wang, N. Dong, W. Li 和 J. Cao, “人工智能的边缘计算: 机器学习视角”, *ACM Computing Surveys*, 第 55 卷, 第 9 期, 第 1–35 页, 2023。[14] D. Xu, W. Yin, X. Jin, Y. Zhang, S. Wei, M. Xu 和 X. Liu, “Llmcad: 快速且可扩展的设备端大型语言模型推理”, *arXiv preprint arXiv:2309.04255*, 2023。[15] K. Srivatsa, K. K. Maurya 和 E. Kochmar, “利用多智能体的力量: 从LLM路由中学到的经验”, *arXiv preprint arXiv:2405.00467*, 2024。[16] B. Lin, T. Peng, C. Zhang, M. Sun, L. Li, H. Zhao, W. Xiao, Q. Xu, X. Qiu, S. Li *et al.*, “Infinite-LLM: 用于长上下文的高效LLM服务, 结合DistAttention和分布式KV缓存”, *arXiv preprint*
- [35] 郑立、蒋伟龙、盛毅、庄珊、吴震、庄宇、林哲、李哲、李东、邢峥 *et al.*, “用 MT-Bench 和 Chatbot Arena 评判 LLM 作为裁判”, *Advances in Neural Information Processing Systems*, 第36卷, 2024。[36] 陈志宏、张鸿波、陈贵明、王天南、张振、李建权、李海洲, “LLM Zoo: 让 ChatGPT 民主化”, <https://github.com/FreedomIntelligence/LLMZoo>, 2023年5月。[37] 沈思、侯亮、周毅、杜楠、Longpre S、魏俊、钟宏伟、Zoph B、Fedus W、陈翔 *et al.*, “专家混合遇上指令调优: 大语言模型的获胜组合”, *arXiv preprint arXiv:2305.14705*, 2023。[38] 缪翔、Oliaro G、张震、程晓、王哲、张哲、Wong R. Y. Y.、朱安、杨立、石翔 *et al.*, “SpecInfer: 通过基于树的推测推理与验证加速大语言模型服务”, 发表于 *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, (ASPLOS 2024)*, 2024, 页932–949。[39] 王阳、林颖、曾曦、张刚, “PrivateLora 用于高效的隐私保护 LLM